Blue Cloud Softech Solutions Ltd.
From Information to *Intelligence*

PHISHING EMAIL ANALYZER

Project Explanation Document

A Python tool that reads a raw **.eml** email file, gathers evidence from its headers, links and attachments, and turns that evidence into an explainable **0–100 risk score** with a **Low / Medium / High / Critical** verdict.

Team Members and Their Parts of the Project

Team member	Role	Part of the project	Sections in this document
Snehith Nomula	Forensics & Integration Lead	• Shared data contract (models.py) • Email parsing and SPF / DKIM / DMARC reading • Sender and authentication indicators • Scoring weight table • Command-line pipeline, integration and web upload interface	• 3.1 Shared data contract • 4.1 – 4.3 Pipeline, parsing, indicators • 5.1 Weight table • 6 Web interface • 7.2 Command-line output (CLI)
Gunnala Kavya	Extraction & Corpus	• Defanged-URL restoration (refang) • URL and attachment extraction with SHA-256 • Text and HTML reports • Synthetic / public sample corpus	• 4.4 URLs and refanging • 4.5 Attachments and SHA-256 • 7.1 The HTML report • 8 Evaluation corpus (part) • 11.2 Implementation note
Benduri Akshaya	Threat Intel & Metrics	• Reputation layer: offline blocklist and VirusTotal client • Scoring engine, buckets and verdict floors • Summary CSV • False-positive / false-negative evaluation	• 4.6 Reputation • 5.2 Scoring engine and verdicts • 7.2 Summary CSV and metrics • 8 Evaluation results, 8.1, 8.2

The project was divided among three members. Each section of this document that belongs to one member carries that member's name directly under its heading; sections marked "All members" are joint work.

Item	Detail
Project	Phishing Email Analyzer (Lab 05 — Python / Email Security)
Organisation	BlueCloud Softech Solutions
Interfaces	Command line (`analyzer.py`) and local Flask web upload page (`webapp.py`)
Basis of this document	The team's project documentation plus a direct read of the project source code. Every code snippet is quoted from the source files; screenshots were captured by running the web application.

Contents

1. Project at a glance
2. Background concepts
3. System architecture
4. Stage-by-stage walkthrough with code
5. Risk scoring and verdicts
6. The web interface (upload UI)
7. Reports and output files
8. Evaluation results
9. Testing strategy
10. Security and safety design
11. Limitations, implementation notes and future work
12. How to run the project
Appendix A. Project structure

1. Project at a glance

> **Team member: All members** — joint section

1.1 What the project does

Phishing emails imitate trusted organisations, but the technical details they cannot fake well — the real sending domain, the Reply-To address, the authentication results, the link targets, the attachment type — are stored inside the email itself. The Phishing Email Analyzer extracts those details from a saved `.eml` file, checks them against rules and reputation data, and reports **which specific observations** raised the risk and by how much.

The tool is deliberately **explainable**. It does not return an opaque "phishing / not phishing" label. Each triggered indicator carries a weight, a plain-English explanation and the raw evidence (for example the exact `From:` header or URL), and the final score is simply the sum of those weights.

1.2 Problem it addresses

A sender name or a keyword check is not enough. A message can look perfectly legitimate to a person while its Reply-To domain differs from the sender, its SPF/DKIM/DMARC results report failures, its link points to a bare IP address, or its attachment is an executable. The project turns an email into structured evidence and combines independent signals into one score.

1.3 Objectives

- Parse `.eml` files safely, even when they are malformed or hostile.
- Extract sender, routing, authentication, body and MIME information.
- Extract and normalise URLs from text and HTML content, including "defanged" links such as `hxxps://evil[.]com`.
- Detect attachments and compute a SHA-256 hash **without** executing, decoding or writing the file to disk.
- Detect sender and authentication anomalies: display-name spoofing, Reply-To / Return-Path mismatch, reported SPF / DKIM / DMARC failures, free-mail impersonation.
- Look up URL and attachment-hash reputation using either an offline blocklist or the VirusTotal API.
- Combine indicators with documented weights and classify the email as Low, Medium, High or Critical.
- Produce HTML and text reports plus CSV / JSON results, and measure accuracy on a synthetic corpus.

1.4 Scope

The tool performs **static analysis of sanctioned `.eml` samples**. It is not an email gateway, a mail-server replacement, a malware sandbox or a machine-learning classifier. It reads the SPF/DKIM/DMARC verdicts that a receiving mail server already wrote into the `Authentication-Results` header; it does **not** itself query DNS or verify DKIM signatures cryptographically.

1.5 Technology stack

Technology	Role in the project
Python 3	Implementation language (project tested on 3.11 and 3.14 according to the README).
`email` package (standard library)	Parses headers, MIME parts and bodies. Uses the `compat32` policy for tolerance of malformed input.
`urlextract` 1.9.0	Intended URL finder; the code falls back to a regular expression if it cannot be used (see Section 11.2).
`hashlib`	SHA-256 fingerprints of attachments, computed in memory.
`requests` 2.34.2	HTTP client for the VirusTotal v3 API.
`python-dotenv` 1.2.3	Loads `VIRUSTOTAL_API_KEY` from a git-ignored `.env` file.
Flask 3.1.3	Local web front end for uploading `.eml` files.
JSON / CSV	Blocklist, VirusTotal cache, results, summary and metrics files.
pytest, pytest-socket	Automated tests, with network access blocked so tests can never spend VirusTotal quota.

2. Background concepts

> **Team member: All members** — joint section

These short definitions make the rest of the document easier to follow.

Term	Meaning
`.eml` file	A saved email in standard RFC 822 text format: a block of headers, then the body, possibly split into MIME parts (plain text, HTML, attachments).
Display name vs. address	In `From: Microsoft 365 Security <no-reply@login.example>` the display name is what the user sees; the address domain is where it really came from. Phishers control the display name freely.
Reply-To / Return-Path	Reply-To is where a reply will go; Return-Path is the envelope address for bounces. Either pointing to a different domain than From is a warning sign.
SPF, DKIM, DMARC	Email-authentication mechanisms. Receiving servers record their verdict (pass, fail, softfail, …) in the `Authentication-Results` header, which this project reads.
Indicator	One triggered piece of evidence with a code, weight, explanation and raw evidence.
Defanging / refanging	Security corpora write links as `hxxps://evil[.]com` so nobody clicks them by accident. Refanging converts them back to `https://evil.com` so they can be analysed.
Punycode host	A hostname beginning `xn--`, the ASCII form of a name with non-ASCII characters; used for look-alike (homograph) domains.
Reputation lookup	Asking a threat-intelligence source whether a URL or a file hash is known to be malicious.

3. System architecture

The system is a linear pipeline. A front end (web page or command line) hands one `.eml` file to a single function, `analyze_one()`, which passes a shared data object through each stage. Each stage adds its findings to that object; the final stage computes the score. Because parsing, extraction, reputation and

scoring are separate modules, each can be tested alone and the reputation source can be swapped without touching the rest.

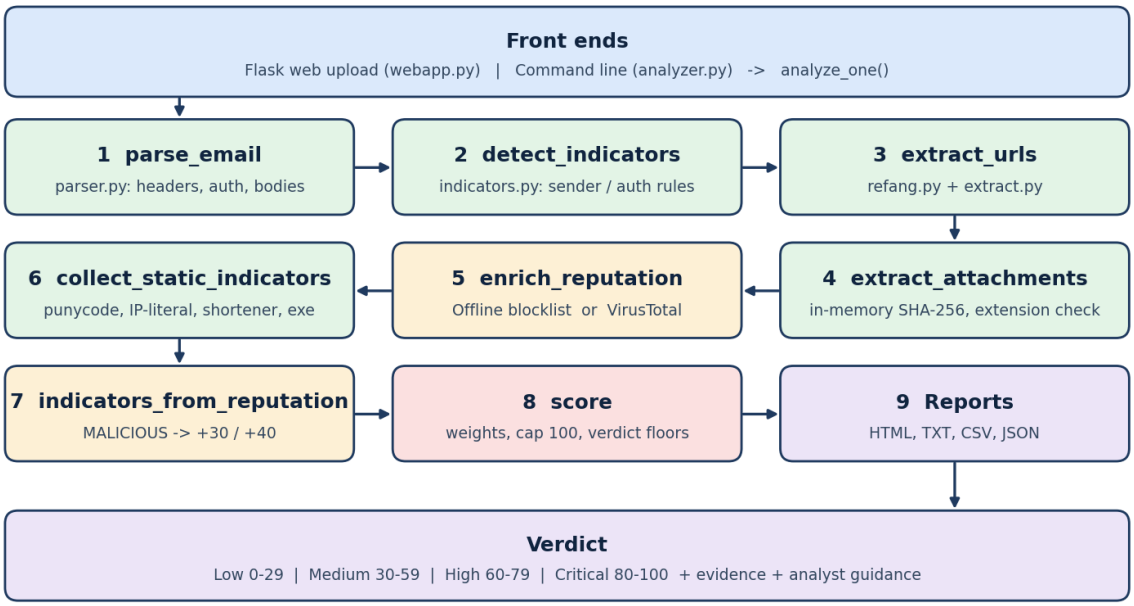


Figure 1. *The analysis pipeline, in the exact order executed by analyze_one(). Stages 2, 3, 4 and 6 need no network; only stage 5 can (optionally) use VirusTotal.*

3.1 The shared data contract

> **Team member: Snehith Nomula** — Forensics & Integration Lead (models.py)

Every module imports its data shapes from `phishanalyzer/models.py`, a file the team froze early so that three people could work in parallel. A test (`tests/test_contract.py`) snapshots the field names and fails if they drift. The central object is `EmailAnalysis`, which starts nearly empty after parsing and is filled in stage by stage:

Code Snippet 1 — *EmailAnalysis — everything known about one email* `[phishanalyzer/models.py]`

```
@dataclass
class EmailAnalysis:
    """Everything known about one analysed .eml file.

    Produced by ``parser.parse_email``, populated by the extraction and
    indicator modules, and finalised in place by ``scoring.engine.score``.
    """

    path: str
    parsed_ok: bool = True
    parse_error: str = ""

    sender: SenderInfo = field(default_factory=SenderInfo)
    indicators: list[Indicator] = field(default_factory=list)
    urls: list[UrlFinding] = field(default_factory=list)
    attachments: list[AttachmentFinding] = field(default_factory=list)
    body_text: str = ""
    body_html: str = ""

    # -- Filled in by scoring.engine.score() --
    base_score: int = 0  # sum of weights, before any verdict floor
    final_score: int = 0  # after cap and floor, 0-100
    verdict: Verdict = Verdict.LOW
```

The enums in the same file encode important design decisions. For example, `AuthStatus` has both `NONE` and `ABSENT`, and `ReputationStatus` has `UNKNOWN` and `UNAVAILABLE`; all four score zero, because "no result" or "lookup impossible" is never treated as evidence of malice.

3.2 Main components

> **Team member: All members** — joint section

File / package	Responsibility
`analyzer.py`	Thin command-line wrapper; delegates to `phishanalyzer.cli.main`.
`webapp.py`	Launcher for the Flask web app (`python webapp.py`).
`phishanalyzer/cli.py`	Argument parsing, file discovery, provider selection, the `analyze_one()` pipeline, output writing.
`phishanalyzer/models.py`	Frozen shared dataclasses and enums.
`phishanalyzer/parser.py`	Reads the `.eml`: sender fields, Received chain, authentication results, text/HTML bodies.
`phishanalyzer/indicators.py`	Header-derived rules (spoofing, mismatches, authentication failures, free-mail lookalike) and conversion of malicious reputation into indicators.
`phishanalyzer/branding.py`	Curated name/logo/team data shared by the web form and the HTML report.
`phishanalyzer/refang.py`	Reverses defanged URL notation.
`phishanalyzer/extract.py`	URL extraction, attachment extraction, SHA-256, URL/attachment indicators.
`phishanalyzer/reputation/`	`base.py` (provider interface), `offline.py` (blocklist/mock), `virustotal.py` (API client with rate limit and cache), `__init__.py` (`enrich_reputation`).
`phishanalyzer/scoring/`	`weights.py` (weight table and verdict floors), `engine.py` (scoring and bucketing).
`phishanalyzer/report/`	`html.py`, `text.py`, `csv_summary.py`.
`phishanalyzer/webapp.py`	Flask application factory, upload form, `/analyze` and `/health` routes, rate limiter.
`tools/`	`make_samples.py` (synthetic corpus), `fetch_corpus.py` (public corpus tooling), `evaluate.py` (metrics).

4. Stage-by-stage walkthrough with code

4.1 The pipeline function

> **Team member: Snehith Nomula** — Forensics & Integration Lead (cli.py)

`analyze_one()` in `cli.py` is the heart of the project. Both the command line and the web application call it, so they can never disagree about a verdict. The order matters: header indicators need only headers, so they run early; the reputation-based indicator can only be raised after the lookup has happened.

Code Snippet 2 — *analyze_one() — the eight stages* `[phishanalyzer/cli.py]`

```
def analyze_one(path: str | Path, provider: ReputationProvider) -> EmailAnalysis:
# ...
    a = parse_email(str(path))
    if not a.parsed_ok:
        return a
```

```
    detect_indicators(a)
    extract_urls(a)
    extract_attachments(a)
    enrich_reputation(a, provider)
    collect_static_indicators(a)
    indicators_from_reputation(a)
    score(a)
    return a
```

4.2 Stage 1 — Parsing the .eml file

> **Team member: Snehith Nomula** — Forensics & Integration Lead (parser.py)

`parse_email()` opens the file in binary mode and parses it with Python's `email` package using the `compat32` policy, chosen because the strict modern policy raises errors on the malformed headers that phishing emails often contain. The function **never raises**: any failure is recorded on the result (`parsed_ok = False` plus a `parse_error` message) so one bad file cannot abort a whole corpus run.

A file with **no headers at all** — an empty file, a binary blob, a renamed PDF — is treated as a parse failure on purpose. Python's parser would otherwise accept it silently, the file would score 0 and be reported as "Low, release normally", which is the worst possible outcome for a tool making a release-or-quarantine decision.

Code Snippet 3 — *parse_email() — tolerant parsing and the headerless-input guard* `[phishanalyzer/parser.py]`

```
def parse_email(path: str) -> EmailAnalysis:
# ...
    analysis = EmailAnalysis(path=path)

    try:
        with open(path, "rb") as handle:
            msg = BytesParser(policy=policy.compat32).parse(handle)
    except (OSError, ValueError) as exc:
        analysis.parsed_ok = False
        analysis.parse_error = f"could not read file: {exc}"
        return analysis

    if not msg.keys():
        analysis.parsed_ok = False
        analysis.parse_error = (
            "no email headers found. The file may be empty, corrupt, or not an "
            "email message (a .p7s/.pdf attachment or a renamed file will look "
            "like this). Nothing was analysed."
        )
        return analysis
```

From the headers it extracts the display name and address of `From`, the `Reply-To` address, `Return-Path`, `Subject`, `Date`, `Message-ID` and the `Received` chain, decoding RFC 2047 encoded words (`_decode`) so an encoded display name cannot hide a brand from later rules. `_extract_bodies()` collects the plain-text and HTML bodies but **skips attachment parts**, so an attached HTML lure is not mistaken for the visible message body.

Reading SPF / DKIM / DMARC

A regular expression finds `spf=`, `dkim=` and `dmarc=` results inside every `Authentication-Results` header. Where several headers disagree (different mail hops), **the worst result wins**. A mechanism that is simply not mentioned becomes `NONE`; a message with no such header at all becomes `ABSENT`. Neither counts as a failure.

Code Snippet 4 — *_parse_auth_results() — folding authentication headers into one verdict per mechanism*
`[phishanalyzer/parser.py]`

```
_AUTH_FIELD_RE = re.compile(
    r"\b(?P<mech>spf|dkim|dmarc)\s*=\s*(?P<result>[a-z]+)", re.IGNORECASE
)

# Result words that mean "this check failed". softfail included: SPF softfail
# means the domain published no SPF record, which is an authorisation gap even
# though it is not an outright forgery. See scoring/weights.py.
_FAIL_RESULTS = {"fail", "softfail", "permerror", "temperror"}
# ...
    if not raw_values:
        return AuthResults(
            header_present=False,
            spf=AuthStatus.ABSENT,
            dkim=AuthStatus.ABSENT,
            dmarc=AuthStatus.ABSENT,
        )

    results: dict[str, str] = {}
    for value in raw_values:
        for match in _AUTH_FIELD_RE.finditer(value or ""):
            mech = match.group("mech").lower()
            result = match.group("result").lower()
            previous = results.get(mech)
            if previous is None or (
                result in _FAIL_RESULTS and previous not in _FAIL_RESULTS
            ):
                results[mech] = result

    def status_for(mech: str) -> AuthStatus:
        raw = results.get(mech)
        if raw is None:
            return AuthStatus.NONE
        if raw in _FAIL_RESULTS:
            return AuthStatus.SOFTFAIL if raw == "softfail" else AuthStatus.FAIL
        if raw == "pass":
            return AuthStatus.PASS
        if raw == "neutral":
            return AuthStatus.NEUTRAL
        return AuthStatus.NONE

    return AuthResults(
        header_present=True,
        spf=status_for("spf"),
        dkim=status_for("dkim"),
        dmarc=status_for("dmarc"),
        raw=" | ".join(_decode(v) for v in raw_values),
    )
```

4.3 Stage 2 — Sender and authentication indicators

> **Team member: Snehith Nomula** — Forensics & Integration Lead (indicators.py)

`detect_indicators()` in `indicators.py` runs five header rules over the parsed sender information. Each rule creates an `Indicator` whose weight comes from the central weight table.

Rule	When it fires
Display-name spoof	The display name contains a brand that phishers commonly impersonate (Microsoft, PayPal, DHL, Chase, HMRC, … — a curated list), but that brand does not appear in the actual sending domain. Names are normalised (case, accents, punctuation stripped) so `Micros0ft`-style and accented variants still match.
Free-mail lookalike	The address is at a consumer webmail provider (gmail.com, outlook.com, …) while the display name claims a corporate function (payroll, HR, help desk, billing, CEO, …).

Rule	When it fires
Reply-To mismatch	The Reply-To domain is a different organisation from the From domain. A subdomain relationship is treated as the same organisation, to avoid false positives.
Return-Path mismatch	The envelope Return-Path domain differs from the From domain (same organisation test as above).
SPF / DKIM / DMARC failure	The reported result is an explicit `fail` or `softfail`. `pass`, `neutral`, `none` and an absent header do not fire.

Code Snippet 5 — *Display-name spoof rule — fires only if the claimed brand is missing from the sending domain* `[phishanalyzer/indicators.py]`

```
def _check_display_name_spoof(sender, out: list[Indicator]) -> None:
    brand = _matched_brand(sender.display_name)
    if not brand or not sender.from_domain:
        return
    if brand in _fold(sender.from_domain):
        return
    out.append(
        _ind(
            IndicatorCode.DISPLAY_NAME_SPOOF,
            f"Display name claims '{brand}' but the message was sent from "
            f"'{sender.from_domain}', which is not that organisation's domain.",
            f"From: {sender.display_name} <{sender.from_addr}>",
        )
    )
```

Code Snippet 6 — *_same_org() — a dot-boundary test that blocks look-alike suffix tricks* `[phishanalyzer/indicators.py]`

```
def _same_org(left: str, right: str) -> bool:
# ...
    right = (right or "").lower().strip(".")
    if not left or not right:
        return False
    return (
        left == right
        or left.endswith("." + right)
        or right.endswith("." + left)
    )
```

Code Snippet 7 — *Authentication rule — only an explicit fail/softfail raises an indicator* `[phishanalyzer/indicators.py]`

```
def _check_auth(sender, out: list[Indicator]) -> None:
    auth = sender.auth
    checks = (
        (auth.spf, IndicatorCode.AUTH_SPF_FAIL, "SPF",
         "the domain publishes no SPF record for the sending host"),
        (auth.dkim, IndicatorCode.AUTH_DKIM_FAIL, "DKIM",
         "the signature was present but did not validate"),
        (auth.dmarc, IndicatorCode.AUTH_DMARC_FAIL, "DMARC",
         "the sending domain's own DMARC policy rejects this message"),
    )
    for status, code, label, consequence in checks:
        if status in (AuthStatus.FAIL, AuthStatus.SOFTFAIL):
            verb = "failed" if status is AuthStatus.FAIL else "soft-failed"
            out.append(
                _ind(
                    code,
                    f"{label} {verb}: {consequence}. An absent header is not "
                    "treated as a failure, only an explicit result is.",
                    auth.raw[:200],
                )
            )
```

4.4 Stage 3 — URL extraction and refanging

> **Team member: Gunnala Kavya** — Extraction & Corpus (refang.py, extract.py)

URLs are collected from three places: the plain-text body, the HTML body, and the visible text of the HTML with tags removed. Links inside HTML `href` attributes are captured because phishing lures often show one domain as link text while pointing the real link elsewhere.

Refanging happens before extraction. No URL finder recognises `hxxps://evil[.]example`, so matching first and refanging afterwards would find nothing on a defanged corpus — the report would look confidently empty rather than obviously broken. `refang.py` applies an ordered list of regular-expression rewrites:

Code Snippet 8 — *refang() — ordered rewrites that restore hxxps, [.], [dot], [at] and similar notation (excerpt)* `[phishanalyzer/refang.py]`

```
_REWRITES: tuple[tuple[re.Pattern[str], str], ...] = (
    (re.compile(r"\[\s*:\s*/\s*/\s*\]", re.IGNORECASE), "://"),
# ...
    (re.compile(r"\bhxxps(?=\s*:)", re.IGNORECASE), "https"),
    (re.compile(r"\bhxxp(?=\s*:)", re.IGNORECASE), "http"),
# ...
    (re.compile(r"\[\s*\.\s*\]"), "."),
    (re.compile(r"\(\s*\.\s*\)"), "."),
# ...
    (re.compile(r"(?<=/)\.+"), ""),
)
# ...
def refang(text: str) -> str:
# ...
    if not text:
        return text
    result = text
    for pattern, replacement in _REWRITES:
        result = pattern.sub(replacement, result)
    return result
```

Each candidate is then normalised (trailing punctuation trimmed, unbalanced brackets removed), filtered to the schemes `http`, `https` and `ftp`, de-duplicated, and capped at **25 URLs per email** to protect the VirusTotal quota. For each URL the extractor records the host and three structural flags that later become indicators:

Code Snippet 9 — *Structural URL checks — IP-literal and Punycode hosts* `[phishanalyzer/extract.py]`

```
def _is_ip_literal(host: str) -> bool:
    if not host:
        return False
    candidate = host[1:-1] if host.startswith("[") else host
    try:
        ipaddress.ip_address(candidate)
    except ValueError:
        return False
    return True



def _is_punycode(host: str) -> bool:
    """True for an xn-- host, i.e. a non-ASCII name in its encoded form."""
    return any(label.startswith("xn--") for label in host.split("."))
```

A URL is flagged as **shortened** if its host is in a fixed list of shortener services (bit.ly, tinyurl.com, t.co, and others). The shortener is flagged, not expanded, because expanding it would mean fetching an attacker-controlled address from the analysis machine.

Code Snippet 10 — *url_indicators() — turning URL flags into scored indicators* `[phishanalyzer/extract.py]`

```
def url_indicators(a: EmailAnalysis) -> list[Indicator]:
```

```
    """Indicators from URL structure, which needs no network access.

    Separate from the reputation-derived indicators because these are provable
    from the URL string alone and so are available even with no API key.
    """
    out: list[Indicator] = []
    for url in a.urls:
        if url.is_punycode:
            out.append(
                _ind(
                    IndicatorCode.URL_PUNYCODE_HOST,
                    f"URL host '{url.host}' is punycode-encoded, meaning it "
                    "contains non-ASCII characters chosen to imitate ASCII. "
                    "This is a deliberate homograph technique.",
                    url.url,
                )
            )
        if url.is_ip_literal:
            out.append(
                _ind(
                    IndicatorCode.URL_ON_IP_LITERAL,
                    f"URL points at the bare IP address '{url.host}' rather "
                    "than a domain name. No legitimate business sender links "
                    "to itself this way.",
                    url.url,
                )
            )
        if url.is_shortener:
            out.append(
                _ind(
                    IndicatorCode.SHORTENED_URL,
                    f"URL uses the shortener '{url.host}', so the true "
                    "destination is not visible in the message.",
                    url.url,
                )
            )
```

4.5 Stage 4 — Attachments and SHA-256

Team member: Gunnala Kavya — Extraction & Corpus (extract.py)

`extract_attachments()` walks the MIME tree. A part counts as an attachment if its disposition is `attachment`, or if it carries a filename and is not explicitly inline. For each attachment the tool records the filename, content type, size, extension and a **SHA-256 digest computed in memory in 64 KB chunks**. Nothing is written to disk, decoded into a file, opened or executed. At most **10 attachments per email** are processed.

Code Snippet 11 — *_digest() — chunked in-memory SHA-256* `[phishanalyzer/extract.py]`

```
def _digest(payload: bytes) -> str:
    """SHA-256 over a payload, chunked so a large attachment never
    materialises as a second full copy in memory."""
    digest = hashlib.sha256()
    view = memoryview(payload)
    for start in range(0, len(view), 65536):
        digest.update(view[start : start + 65536])
    return digest.hexdigest()
```

Code Snippet 12 — *extract_attachments() — the per-attachment loop* `[phishanalyzer/extract.py]`

```
            break


        try:
            payload = part.get_payload(decode=True)
        except Exception:  # noqa: BLE001, S112 - hostile input, never let it escape
            # A part that cannot be decoded is skipped, not fatal: one corrupt
            # attachment must not lose the other nine findings for the email.
            continue
        if payload is None:
            continue
```

```
        index += 1
        filename = part.get_filename() or f"attachment-{index}"
        ext = _ext_of(filename)
        dangerous = ext in DANGEROUS_EXTENSIONS

        a.attachments.append(
            AttachmentFinding(
                filename=filename,
                content_type=part.get_content_type(),
                size_bytes=len(payload),
                sha256=_digest(payload),
                ext=ext,
                is_dangerous_ext=dangerous,
                reputation=ReputationStatus.UNAVAILABLE,
            )
        )

    if truncated:
        _note_cap(a, f"attachment extraction truncated at {MAX_ATTACHMENTS} (cap)")
```

An attachment is marked as having a **dangerous extension** when its extension belongs to a fixed set that covers executables and scripts (`exe`, `scr`, `bat`, `cmd`, `ps1`, `vbs`, `js`, `hta`, `jar`, `lnk`, …), active web content (`html`, `htm`, `svg`, `mhtml`, and `eml`), disk images (`iso`, `img`, `vhd`), macro-enabled Office formats (`docm`, `xlsm`, `pptm`, …) and mobile packages (`apk`, `ipa`). A `.html` attachment is included deliberately: it can render a complete fake login page locally and bypass web filtering.

4.6 Stage 5 — Reputation (offline or VirusTotal)

> **Team member: Benduri Akshaya** — Threat Intel & Metrics (reputation/)

Extraction never performs lookups; it leaves every URL and attachment with status `UNAVAILABLE`. The reputation layer fills that in afterwards through a common interface with two interchangeable implementations.

Code Snippet 13 — *ReputationProvider — the interface both providers implement* `[phishanalyzer/reputation/base.py]`

```
class ReputationProvider(ABC):
    """Look up the reputation of a URL or a file hash."""

    #: Recorded on every ReputationResult this provider returns, so a report
    #: always says where a verdict came from.
    name: str = "abstract"

    @abstractmethod
    def lookup_url(self, url: str) -> ReputationResult:
        """Reputation for a URL.

        Returns UNAVAILABLE rather than raising when it cannot answer. A
        provider that raises on a transient network error would abort a corpus
        run; a provider that reports UNAVAILABLE lets the run continue and the
        report state plainly that the check did not happen.
        """

    @abstractmethod
    def lookup_hash(self, sha256: str) -> ReputationResult:
        """Reputation for a file SHA-256.

        Same contract as lookup_url. Expects a lowercase hex digest.
        """
```

Offline provider (default)

`OfflineProvider` loads `samples/offline_blocklist.json` (a list of malicious URLs and SHA-256 hashes generated together with the synthetic corpus). A listed item returns `MALICIOUS`; an unlisted item returns `CLEAN`. If **no** blocklist could be loaded, every lookup returns `UNAVAILABLE` instead of `CLEAN` — an unconfigured run must never claim that things were checked and found safe.

Code Snippet 14 — *OfflineProvider.lookup_url() — blocklist hit, mock mode, or honest "unavailable"* `[phishanalyzer/reputation/offline.py]`

```
    def lookup_url(self, url: str) -> ReputationResult:
        key = (url or "").strip().lower()
        if key in self._urls:
            return ReputationResult(
                status=ReputationStatus.MALICIOUS,
                malicious=12,
                suspicious=2,
                total=self.total_engines,
                source=LookupSource.OFFLINE_BLOCKLIST,
            )
        if self.mock:
            return self._mock_result(f"url:{key}")
        if not self._loaded:
            return self._no_list()
        return ReputationResult(
            status=ReputationStatus.CLEAN,
            malicious=0,
            suspicious=0,
            total=self.total_engines,
            source=LookupSource.OFFLINE_BLOCKLIST,
        )
```

VirusTotal provider (opt-in)

`VirusTotalProvider` queries the VirusTotal v3 API for a URL (using its base64 identifier) or a file hash and maps the `last_analysis_stats` counts to a status: any malicious engine → `MALICIOUS`, else any suspicious → `SUSPICIOUS`, else `CLEAN`. A 404 means `UNKNOWN`. Because the free API allows only **4 requests per minute and 500 per day**, the client adds three protections:

- **Rate limiting** — requests are spaced at least 15 seconds apart and a local daily counter stops before the quota is exhausted.
- **Disk cache** — every response is stored under `.vt_cache/`, so a second run over the same items makes no network requests.
- **Honest absence** — no key, HTTP 404, HTTP 429 or any failure returns a result object instead of raising, and scores zero.

Code Snippet 15 — *Throttle and daily quota counter* `[phishanalyzer/reputation/virustotal.py]`

```
    def _throttle(self) -> None:
        """Block until the minimum interval since the last request has passed."""
        with self._lock:
            elapsed = time.monotonic() - self._last_request_at
            if elapsed < self.min_interval:
                time.sleep(self.min_interval - elapsed)
            self._last_request_at = time.monotonic()


    def _consume_quota(self) -> None:
        self._reset_day_if_needed()
        if self._requests_today >= REQUESTS_PER_DAY:
            raise QuotaExhausted(
                f"VirusTotal daily allowance of {REQUESTS_PER_DAY} requests "
                "reached. Re-run with a warm cache, or wait for 00:00 UTC."
            )
        self._requests_today += 1
```

Code Snippet 16 — *_result_from_payload() — mapping VirusTotal statistics to a status*
[phishanalyzer/reputation/virustotal.py]

```
    @staticmethod
    def _result_from_payload(payload: dict | None) -> ReputationResult | None:
# ...
        try:
            stats = payload["data"]["attributes"]["last_analysis_stats"]
            malicious = int(stats.get("malicious", 0))
            suspicious = int(stats.get("suspicious", 0))
            total = int(
                stats.get("malicious", 0)
                + stats.get("suspicious", 0)
                + stats.get("undetected", 0)
                + stats.get("harmless", 0)
            )
        except (KeyError, TypeError, ValueError):
            return None


        if malicious > 0:
            status = ReputationStatus.MALICIOUS
        elif suspicious > 0:
            status = ReputationStatus.SUSPICIOUS
        else:
            status = ReputationStatus.CLEAN


        return ReputationResult(
            status=status,
            malicious=malicious,
            suspicious=suspicious,
            total=total,
            source=LookupSource.VIRUSTOTAL,
        )
```

Finally `enrich_reputation()` applies the provider to each URL and attachment. Every call is wrapped so that any provider failure degrades that single item to `ERROR` (score 0) rather than aborting the run:

Code Snippet 17 — *_safe_lookup() — a provider failure becomes ERROR, never a crash*
[phishanalyzer/reputation/__init__.py]

```
def _safe_lookup(provider: ReputationProvider, url: str | None, sha256: str | None) -> ReputationResult:
# ...
    try:
        if url is not None:
            result = provider.lookup_url(url)
        else:
            result = provider.lookup_hash(sha256 or "")
    except Exception as exc:  # noqa: BLE001 - deliberate catch-all boundary
        return ReputationResult(
            status=ReputationStatus.ERROR,
            error=f"{type(exc).__name__}: {exc}",
            source=_source_of(provider),
        )
    if not isinstance(result, ReputationResult):
        return ReputationResult(
            status=ReputationStatus.ERROR,
            error=f"provider returned {type(result).__name__}, expected ReputationResult",
            source=_source_of(provider),
        )
    return result
```

Afterwards, `indicators_from_reputation()` raises a `malicious_url` or `malicious_attachment` indicator **only** for `MALICIOUS` results. `UNKNOWN`, `UNAVAILABLE` and `ERROR` contribute nothing, so a missing API key can never make an email look more suspicious.

5. Risk scoring and verdicts

5.1 The weight table

> **Team member: Snehith Nomula** — Forensics & Integration Lead (scoring/weights.py)

All weights live in one data-only file, `scoring/weights.py`, so the risk profile can be tuned in one place and re-measured with `tools/evaluate.py`. The five headline weights are those fixed by the lab specification; the others are the team's additions for signals the specification asks to detect but does not score.

Indicator	Weight	Meaning
`malicious_attachment`	+40	Attachment hash reported malicious by the reputation provider.
`malicious_url`	+30	URL reported malicious by the reputation provider.
`freemail_lookalike`	+25	Corporate role claimed from a consumer webmail address.
`dangerous_attachment_extension`	+25	Executable, script, active-content or macro-enabled attachment.
`display_name_spoof`	+20	Brand in display name that is absent from the sending domain.
`auth_spf_fail`	+20	SPF reported fail or softfail.
`auth_dkim_fail`	+20	DKIM reported fail.
`auth_dmarc_fail`	+20	DMARC reported fail.
`reply_to_mismatch`	+15	Reply-To goes to a different organisation than From.
`url_punycode_host`	+15	URL host is Punycode-encoded (`xn--`).
`return_path_mismatch`	+10	Envelope sender differs from From (weaker: legitimate forwarding causes this).
`url_on_ip_literal`	+10	URL points to a bare IP address.
`shortened_url`	+5	URL uses a known shortener (weak on its own).

Code Snippet 18 — *WEIGHTS and VERDICT_FLOORS (comments omitted)* `[phishanalyzer/scoring/weights.py]`

```
WEIGHTS: dict[IndicatorCode, int] = {
    IndicatorCode.DISPLAY_NAME_SPOOF: 20,
    IndicatorCode.REPLY_TO_MISMATCH: 15,
    IndicatorCode.RETURN_PATH_MISMATCH: 10,
    IndicatorCode.AUTH_SPF_FAIL: 20,
    IndicatorCode.AUTH_DKIM_FAIL: 20,
    IndicatorCode.AUTH_DMARC_FAIL: 20,
    IndicatorCode.FREEMAIL_LOOKALIKE: 25,
    IndicatorCode.DANGEROUS_ATTACHMENT_EXT: 25,
    IndicatorCode.SHORTENED_URL: 5,
    IndicatorCode.URL_PUNYCODE_HOST: 15,
    IndicatorCode.URL_ON_IP_LITERAL: 10,
    IndicatorCode.MALICIOUS_URL: 30,
    IndicatorCode.MALICIOUS_ATTACHMENT: 40,
}
# ...
VERDICT_FLOORS: dict[IndicatorCode, Verdict] = {
    IndicatorCode.MALICIOUS_ATTACHMENT: Verdict.HIGH,
    IndicatorCode.MALICIOUS_URL: Verdict.MEDIUM,
    IndicatorCode.AUTH_DMARC_FAIL: Verdict.MEDIUM,
}
```

5.2 Scoring engine and verdict buckets

> **Team member: Benduri Akshaya** — Threat Intel & Metrics (scoring/engine.py) | **Snehith Nomula** — Forensics & Integration Lead (floor table in weights.py)

`score()` sums the weights of every triggered indicator (`base_score`), caps the total at 100 (`final_score`) and maps it to a bucket:

Score	Verdict	General interpretation
0–29	Low	Few or no significant suspicious indicators. Report guidance: release normally.
30–59	Medium	Some evidence needs analyst attention. Hold for manual review.
60–79	High	Multiple or stronger indicators. Quarantine pending review.
80–100	Critical	Strong combination of indicators. Quarantine, block sender, escalate.

Code Snippet 19 — *bucket() — score to verdict* `[phishanalyzer/scoring/engine.py]`

```
def bucket(n: int) -> Verdict:
# ...
    if n < 0:
        n = 0
    elif n > 100:
        n = 100
    if n < 30:
        return Verdict.LOW
    if n < 60:
        return Verdict.MEDIUM
    if n < 80:
        return Verdict.HIGH
    return Verdict.CRITICAL
```

Verdict floors

The lab specification is internally inconsistent: it gives a confirmed malicious attachment a weight of +40 (which lands in Medium) yet also requires such a single strong indicator to reach at least High. The team kept the specified weights unchanged and added a separate **verdict floor** table: certain indicators guarantee a minimum verdict no matter how few other signals are present.

Indicator	Minimum verdict	Effect
`malicious_attachment`	High	Final score is raised to at least 60.
`malicious_url`	Medium	Final score is raised to at least 30.
`auth_dmarc_fail`	Medium	Final score is raised to at least 30.

When a floor applies, the engine raises **both** the verdict and the numeric score, so the report never shows a contradiction such as “40/100 — High”. The untouched sum remains visible as `base_score`, and `floor_applied` records that a floor was used.

Code Snippet 20 — *score() — sum, cap, bucket, then apply the highest applicable floor* `[phishanalyzer/scoring/engine.py]`

```
    a.base_score = sum(WEIGHTS.get(i.code, 0) for i in a.indicators)
    a.final_score = min(a.base_score, 100)
    a.verdict = bucket(a.final_score)
    a.floor_applied = None
```

```
    floors = [VERDICT_FLOORS[i.code] for i in a.indicators if i.code in VERDICT_FLOORS]
    if not floors:
        return

    highest = max(floors, key=_severity)
    if _severity(highest) > _severity(a.verdict):
        a.verdict = highest
        a.floor_applied = highest
        a.final_score = max(a.final_score, _FLOOR_SCORE[highest])
        a.final_score = min(a.final_score, 100)
```

5.3 Worked example from the corpus

> **Team member: All members** — joint section

The synthetic sample `phish-01-m365-password-expiry.eml` is a fake Microsoft password-expiry message. Its raw headers, exactly as stored in the sample file:

Code Snippet 21 — *Key headers and link of the sample email* `[samples/synthetic/phishing/phish-01-m365-password-expiry.eml]`

```
From: Microsoft 365 Security <no-reply@login.microsoft.example>
Reply-To: reset@microsoft-support.example
Return-Path: <bounce@login.microsoft.example>
Subject: Action required: your password expires in 24 hours
Authentication-Results: mx.example; spf=fail (...); dkim=fail; dmarc=fail
...
https://login.microsoft.example/renew
```

Running the analyzer produces these indicators (taken from `output/results.json`):

Indicator	Weight	Evidence
`malicious_url`	+30	Link is on the offline blocklist (12/70 engines in the offline record).
`auth_spf_fail`	+20	`spf=fail` in Authentication-Results.
`auth_dkim_fail`	+20	`dkim=fail`.
`auth_dmarc_fail`	+20	`dmarc=fail`.
`reply_to_mismatch`	+15	Reply-To domain `microsoft-support.example` differs from `login.microsoft.example`.
Base score	**105**	Sum of the weights.
Final score	**100**	Capped at 100 → **Critical**.

Two details are worth noticing. First, no display-name-spoof indicator fired even though the name says “Microsoft”, because the word `microsoft` appears in the sending domain — the rule only fires when the claimed brand is missing from the domain. Second, the raw sum (105) exceeds 100, which is why `base_score` is stored separately from `final_score`.

6. The web interface (upload UI)

> **Team member: Snehith Nomula** — Forensics & Integration Lead (webapp.py, branding.py)

In addition to the command line, the project includes a small Flask web application (`phishanalyzer/webapp.py`, launched with `python webapp.py`) so that an analyst can simply upload an

`.eml` file in a browser. The web layer is a **thin adapter**: it calls the same `analyze_one()` and `render_html()` functions as the command line and contains no analysis logic of its own. The screenshots below were captured from the running application.

6.1 The upload page

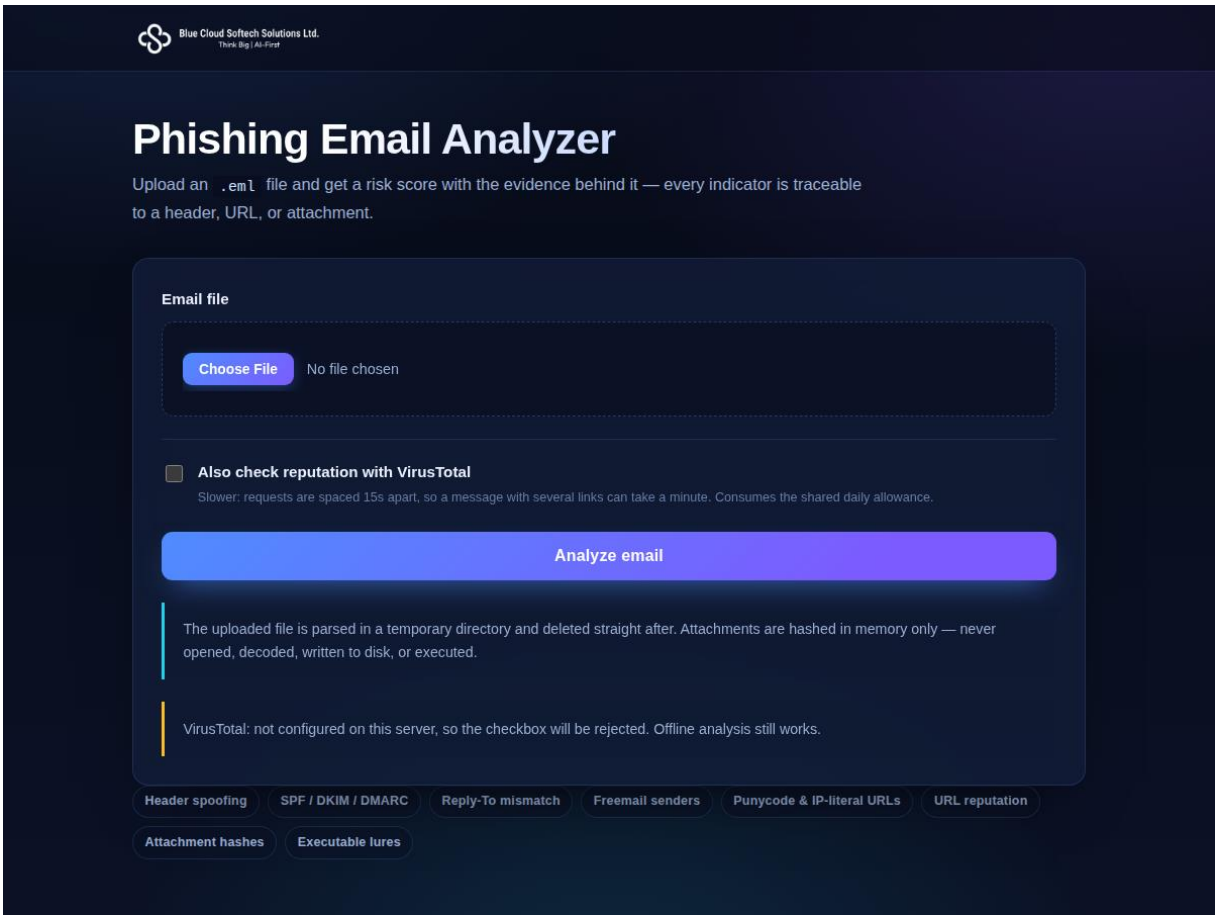


Figure 2. *Landing page at http://127.0.0.1:8000/: file chooser, the optional VirusTotal checkbox, the "analysis is local" notice and the VirusTotal status note.*

The page contains the following elements:

- **Header bar** with the organisation logo (falls back to a text wordmark if the logo file is missing).
- **Email file** chooser restricted to `.eml` / `message/rfc822` (`accept=".eml,message/rfc822"`, marked `required`).
- **"Also check reputation with VirusTotal"** checkbox, unticked by default, with a warning that requests are spaced 15 s apart and consume the shared daily allowance.
- **Analyze email** button, which submits the form as `multipart/form-data` to `/analyze`.
- **Privacy note**: the file is parsed in a temporary directory and deleted straight after; attachments are hashed in memory only.
- **VirusTotal status note**: shows whether a key is configured and how many of the 500 daily lookups have been used (amber when not configured).
- **Feature chips** listing what is checked, and a **"Built by"** footer with the three team members.

Code Snippet 22 — *The upload form markup (inside the _FORM template)* `[phishanalyzer/webapp.py]`

```
    <form method="post" action="{{ url_for('analyze') }}" enctype="multipart/form-data">
      <label for="file">Email file</label>
```

```
    <div class="drop">
      <input type="file" id="file" name="file" accept=".eml,message/rfc822" required>
    </div>

    <div class="opt">
      <input type="checkbox" id="virustotal" name="virustotal" value="1">
      <label for="virustotal">
        Also check reputation with VirusTotal
        <small>Slower: requests are spaced 15s apart, so a message with several
        links can take a minute. Consumes the shared daily allowance.</small>
      </label>
    </div>

    <button type="submit">Analyze email</button>
```

6.2 Selecting a file and submitting

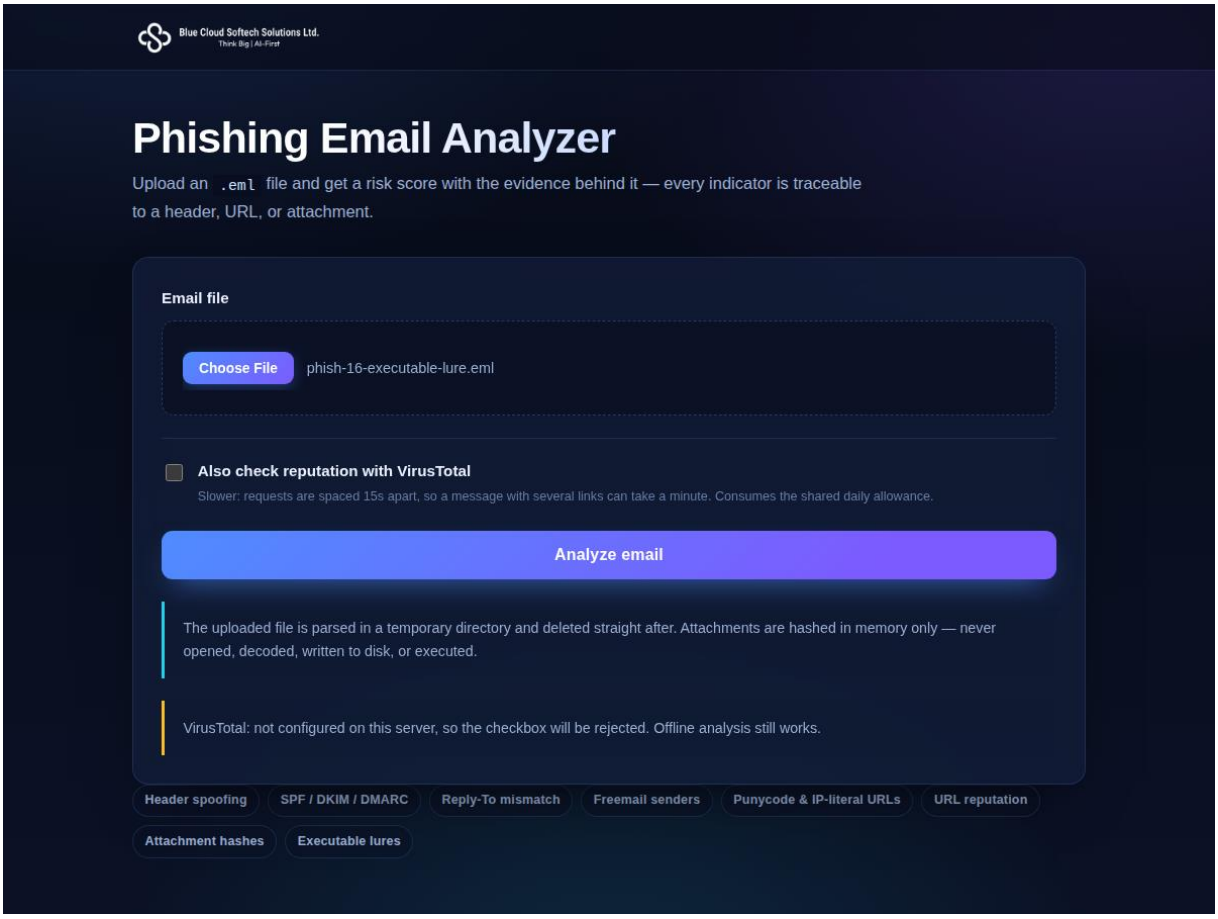


Figure 3. *The same page after choosing phish-16-executable-lure.eml. Clicking "Analyze email" runs the pipeline and returns the report.*

6.3 What happens on the server

When the form is posted, the `/analyze` route performs these steps in order:

1. **Rate-limit check** — at most 10 uploads per client per 5 minutes; otherwise HTTP 429.
2. **Validate the upload** — no file selected gives HTTP 400 with the message "No .eml file was selected."
3. **Choose the reputation provider** — offline blocklist by default; the shared VirusTotal provider only if the checkbox is ticked. Ticking it with no key configured is refused with HTTP 503 rather than silently falling back to offline.

4. **Save to a temporary directory** under a server-generated name (`upload.eml`); the client's filename never touches the filesystem.
5. **Run** `analyze_one()` — the same pipeline as the CLI. Live VirusTotal analyses are serialised with a lock.
6. **Render the HTML report** and return it as a **file download** named `<name>-report.html`, with `Cache-Control: no-store` and `X-Content-Type-Options: nosniff`.
7. **Delete the temporary directory**, even if analysis raised an exception.

Code Snippet 23 — *The /analyze route (excerpts)* `[phishanalyzer/webapp.py]`

```
        upload = request.files.get("file")
        if upload is None or not upload.filename:
            return _index_with_error("No .eml file was selected."), 400

        wants_vt = bool(request.form.get("virustotal")) and enable_vt
        if request.form.get("virustotal") and not enable_vt:
            return _index_with_error(
                "VirusTotal lookups are disabled on this server."
            ), 400
# ...
            provider = get_vt_provider(cache_dir)
        else:
            provider = OfflineProvider(Path(blocklist), mock=False)

        from tempfile import TemporaryDirectory

        with TemporaryDirectory(prefix="pea-web-") as tmpdir:
            # Server-generated name. The client filename never reaches the
            # filesystem, so a crafted name cannot escape the temp directory.
            target = Path(tmpdir) / "upload.eml"
            upload.save(target)
# ...
        # Report the name the user recognises, not the temp path. Leaving the
        # real path in leaks the server's directory layout into a file that gets
        # shared around.
        original = secure_filename(upload.filename) or "upload.eml"
        analysis.path = original

        html = render_html(analysis)
        stem = Path(original).stem or "email"
        response = Response(html, mimetype="text/html")
        response.headers["Content-Disposition"] = (
            f'attachment; filename="{stem}-report.html"'
        )
        # Belt and braces with the attachment disposition: even if something
        # upstream drops that header, the browser must not sniff this into
        # something executable.
        response.headers["X-Content-Type-Options"] = "nosniff"
        response.headers["Cache-Control"] = "no-store"
        return response
```

6.4 Error states

Errors are shown in a red banner above the form. The screenshot shows the case where the VirusTotal box was ticked but no API key is configured on the server:

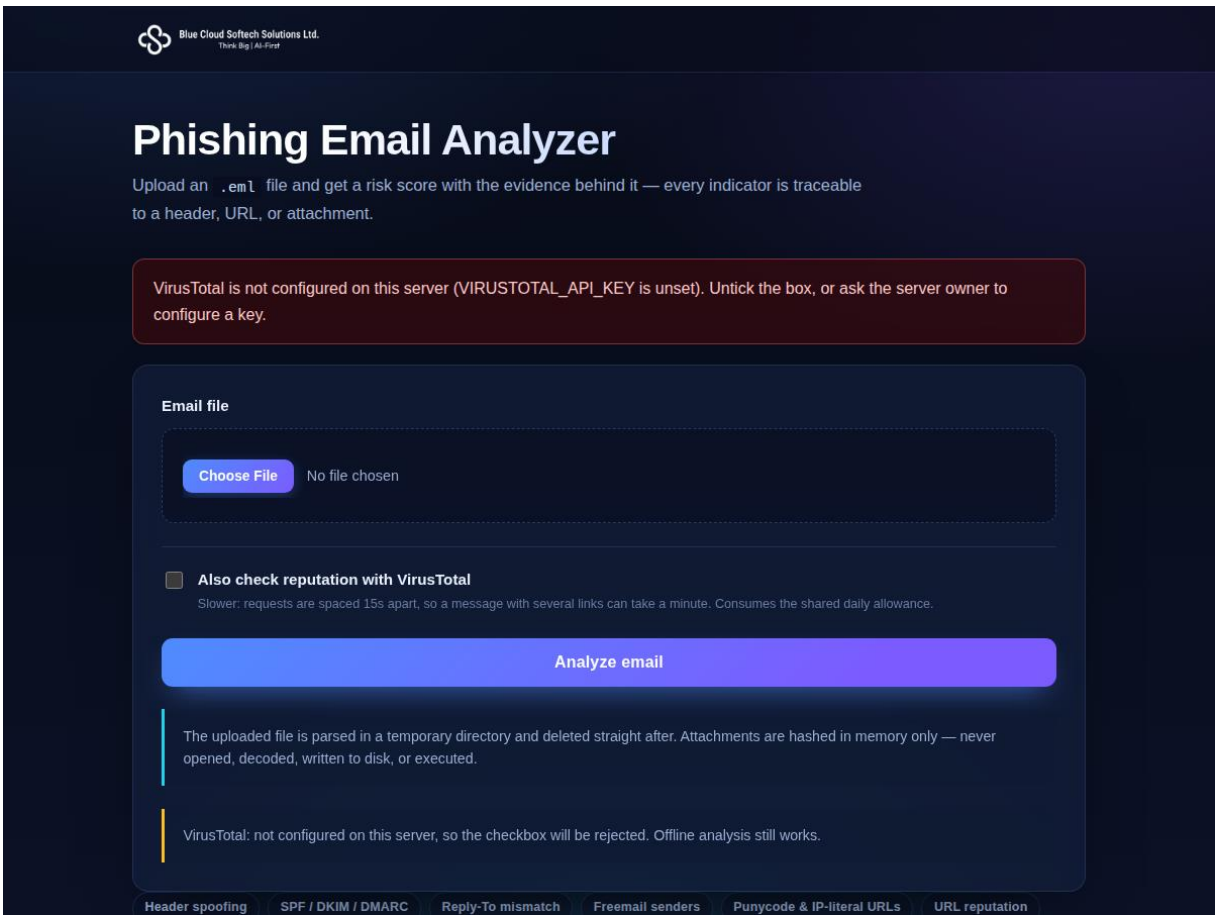


Figure 4. *Error banner: VirusTotal requested but VIRUSTOTAL_API_KEY is not set (HTTP 503).*

Situation	HTTP	Behaviour
No file chosen	400	“No .eml file was selected.”
VirusTotal ticked, no key configured	503	Refuses instead of quietly analysing offline (the user asked for reputation data).
VirusTotal disabled on the server	400	“VirusTotal lookups are disabled on this server.”
Too many uploads	429	Rate-limit message with the configured limit and window.
File over 25 MB	413	Size-limit message.

6.5 Why the report is a download, not a web page

The report is built from attacker-controlled email content. Although `report/html.py` escapes every value and only turns `http`, `https` and `ftp` links into clickable anchors, rendering that document on the same origin as the upload form would turn any future escaping mistake into stored cross-site scripting against exactly the users most likely to be targeted. Returning the report as a download removes that risk.

6.6 Web security controls

The rate limiter is a small fixed-window counter kept in process memory; old entries are pruned so a flood of source addresses cannot grow it without bound:

Code Snippet 24 — *RateLimiter.allow() — per-client fixed-window limit* `[phishanalyzer/webapp.py]`

```
    def allow(self, key: str, now: float) -> bool:
```

```
        with self._lock:
            self._prune(now)
            times = [t for t in self._hits.get(key, ()) if now - t < self.window]
            if len(times) >= self.limit:
                self._hits[key] = times
                return False
            times.append(now)
            self._hits[key] = times
            return True
```

Control	Detail
Local binding	Binds to `127.0.0.1:8000` by default; the launcher prints a warning if bound elsewhere.
Debug off	`debug=False` is hard-coded (the Werkzeug debugger allows remote code execution).
Upload size cap	25 MB (`MAX_CONTENT_LENGTH`).
Rate limit	10 uploads per client per 300 s (configurable); proxy headers are trusted only with `--trust-proxy`.
Offline by default	VirusTotal is opt-in per request so a few uploads cannot drain the 500/day quota.
Single VirusTotal provider	One process-wide provider so the daily counter is global; the app refuses to start with VirusTotal enabled and more than one worker process.
`/health` endpoint	Returns JSON with status, worker count, whether VirusTotal is configured and the quota used.

7. Reports and output files

7.1 The HTML report

Team member: Gunnala Kavya — Extraction & Corpus (report/html.py, report/text.py)

The report returned by the web page (and written by the command line to `output/reports/<name>.html`) is a single self-contained file: the logo is embedded as a data URI so the report still displays correctly when archived or emailed. The figure below is the top of the report for `phish-16-executable-lure.eml`, an email carrying an executable attachment:

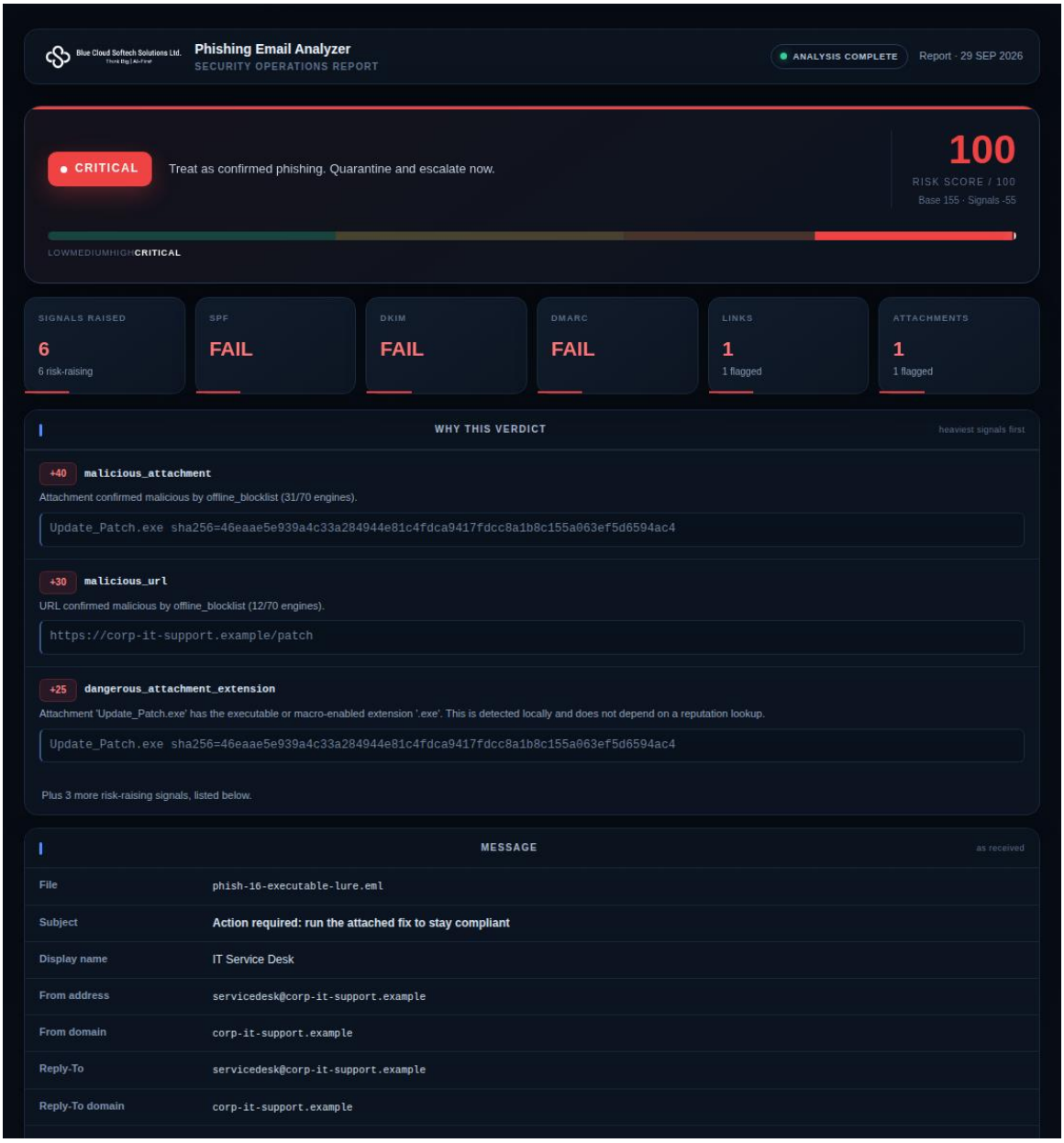


Figure 5. *Top of the HTML report: verdict band (Critical, 100/100), summary tiles for SPF/DKIM/DMARC, links and attachments, and the "Why this verdict" list of heaviest signals.*

The report contains, from top to bottom: the **verdict band** with the score and a bar showing the four ranges; **summary tiles**; **"Why this verdict"** (heaviest signals first, each with weight, explanation and raw evidence); a **message** table (subject, display name, addresses, Reply-To, Return-Path, Date, Message-ID, authentication results); **all indicators**; the **links** found with their reputation badges; the **attachments** with type, size and SHA-256; and a **"What to do"** checklist that depends on the verdict.

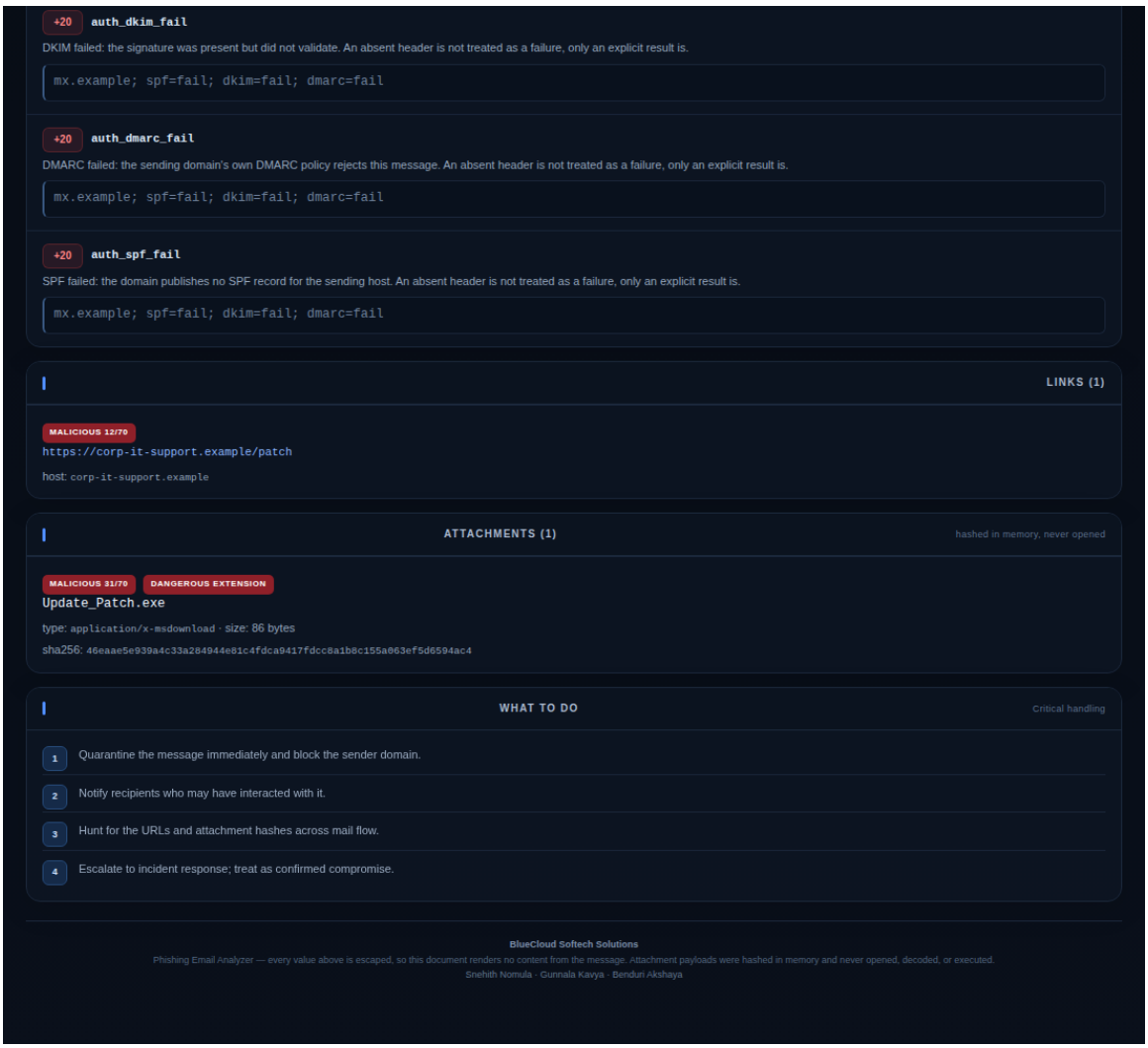


Figure 6. *Lower part of the report: link and attachment tables (with "Malicious" and "Dangerous extension" badges) and the analyst guidance for a Critical verdict.*

Verdict	Analyst guidance printed in the report
Critical	Quarantine immediately and block the sender domain; notify recipients who may have interacted; hunt for the URLs and hashes across mail flow; escalate to incident response.
High	Quarantine pending analyst review; do not release without manual inspection; warn recipients if credentials or payment were requested; check earlier mail from the domain.
Medium	Hold for manual review; inspect the sending domain and authentication results by hand; compare against known-good contacts.
Low	No action required; release normally; recorded for baseline tuning.

7.2 Command-line output

> **Team member: Snehith Nomula** — Forensics & Integration Lead (CLI, cli.py) | **Benduri Akshaya** — Threat Intel & Metrics (summary.csv, metrics.json)

Running `python analyzer.py <files or folders>` analyses every `.eml` found (folders are searched recursively) and writes:

Output	Purpose
`output/reports/<email>.html`	Styled report shown above.

Output	Purpose
`output/reports/<email>.txt`	Plain-text equivalent for terminals and tickets.
`output/summary.csv`	One row per email, sorted by descending score: sender, domains, score, verdict, top indicator, URL/attachment counts, reputation source.
`output/results.json`	Full structured analysis of every email.
`output/metrics.json`	Confusion matrix and rates from `tools/evaluate.py`.
`output/accuracy_writeup.md`	Written interpretation of the evaluation results.

The command-line provider choice is conservative: **only** an explicit `--virustotal` flag can spend API quota. Otherwise the local blocklist is used and no network request is made; `--offline` goes further and disables reputation entirely (all lookups report `UNAVAILABLE`).

8. Evaluation results

> **Team member: Gunnala Kavya** — Extraction & Corpus (sample corpus) | **Benduri Akshaya** — Threat Intel & Metrics (evaluation, evaluate.py)

The project includes a deterministic synthetic corpus generated by `tools/make_samples.py`: **40 messages — 20 phishing and 20 legitimate**. The generator also writes a ground-truth file and the offline blocklist. `tools/evaluate.py` compares `summary.csv` with the ground truth. For the primary measurement, **High and Critical count as a positive (phishing) prediction; Medium does not.** These figures were re-produced during the preparation of this document by re-running the analyzer and evaluator on the supplied corpus; they match the stored `metrics.json`.

Metric	Result	Notes
Total messages	40	20 phishing, 20 legitimate
True positives (TP)	19	Phishing rated High or Critical
False positives (FP)	0	No legitimate mail rated High or Critical
True negatives (TN)	20	All legitimate mail rated Low
False negatives (FN)	1	`phish-03-docusign-signature.eml` (score 50, Medium)
Precision	1.00	TP / (TP + FP)
Recall	0.95	TP / (TP + FN)
False-positive rate	0.00	
False-negative rate	0.05	
Verdict distribution	Low 20 · Medium 1 · High 7 · Critical 12	

With `--include-medium` (Medium also counted as positive) recall becomes 1.00 and false negatives drop to 0.

8.1 The one false negative

> **Team member: Benduri Akshaya** — Threat Intel & Metrics

`phish-03-docusign-signature.eml` is deliberately borderline: SPF and DKIM pass and Reply-To matches, so its only header signal is a DMARC failure (+20) plus a blocklisted URL (+30) = **50 = Medium**. The team kept it in the corpus on purpose, so the reported recall is not measured on data tuned to hide a weak spot. Raising the `malicious_url` floor to High would make the corpus 20/20 but would also raise every other blocklist hit, so the choice was left at Medium.

8.2 How to read these numbers honestly

> **Team member: Benduri Akshaya** — Threat Intel & Metrics

> **Important caveat**
> These results describe the **synthetic** corpus, whose authentication headers and reputation data were written by the team. They show that the pipeline is wired correctly and the rules behave as designed; they are **not** a measure of real-world phishing-detection accuracy. In particular, `auth_dmarc_fail` fires on all 20 phishing samples only because those results were authored into the headers. The corpus is also small, so zero false positives is a weak result.

The accuracy write-up also notes that running the same corpus with `--virustotal` would score lower, because the corpus uses reserved example domains that VirusTotal has no record of; the tool then correctly declines to call them malicious.

9. Testing strategy

> **Team member: All members** — joint section

Who wrote which tests: Snehith Nomula — test_parser, test_indicators, test_contract; Gunnala Kavya — test_refang, test_extract, test_fetch_corpus; Benduri Akshaya — test_reputation, test_scoring, test_reports; test_webapp and test_end_to_end are joint integration tests.

The `tests/` folder contains a test module for each area. A shared fixture blocks all network access (`pytest-socket`), so no test can reach VirusTotal even if a key is configured on the developer's machine.

Test file	What it protects
`test_parser.py`	Header, body and authentication extraction; headerless and malformed input.
`test_refang.py`	Restoration of common defanged URL forms.
`test_extract.py`	URL de-duplication and caps, attachment detection, SHA-256 hashing.
`test_indicators.py`	Sender and authentication anomaly rules.
`test_scoring.py`	Weights, score cap, buckets and verdict floors.
`test_reputation.py`	Offline and VirusTotal provider behaviour using a fake HTTP session.
`test_reports.py`	Text and HTML report generation, including escaping.
`test_webapp.py`	Upload flow and security behaviour of the Flask app (hostile uploads, limits, error codes).
`test_end_to_end.py`	The full pipeline on complete samples.

Test file	What it protects
`test_contract.py`	Guards the frozen data models against accidental field changes.
`test_fetch_corpus.py`	Public-corpus tooling.

Continuous integration (`.github/workflows/ci.yml`) runs the suite and, according to the README, fails the build if `samples/`, `output/` or `.env` is ever committed.

10. Security and safety design

> **Team member: All members** — joint section

- **Attachments are never executed, opened or written to disk** — only hashed in memory.
- **Hostile input is expected**: the parser never raises, and provider failures degrade to a zero-scoring `ERROR`.
- **Absence is never evidence**: missing authentication headers, unknown reputation or an unavailable service all score zero.
- **A file that could not be read is never reported as clean**: unparseable files are reported as unclassified.
- **Secrets stay out of source code**: the VirusTotal key is read from the environment or a git-ignored `.env`.
- **Resource limits**: 25 URLs and 10 attachments per email, a 25 MB upload cap, and a rate limit on uploads.
- **Reproducible, safe test data**: the corpus is synthetic; attacker domains use reserved names (`.example`, `.invalid`, `.test`), so nothing points to live infrastructure.
- **Local service**: the web app is meant for local use and has no authentication; it should not be exposed to the internet.

11. Limitations, implementation notes and future work

11.1 Limitations

> **Team member: All members** — joint section

- The analyzer is rule-based; it does not use machine learning.
- SPF, DKIM and DMARC results are **read from the header**, not independently verified through DNS or signature checks.
- Threat intelligence is only as good as the configured source; new malicious infrastructure may have no record, and “unknown” is not proof of safety.
- The evaluation corpus is small and synthetic.
- Legitimate bulk mail, forwarding services and unusual Reply-To setups can trigger anomalies that need analyst context.
- Static analysis cannot replace a sandbox when real file behaviour matters.

11.2 Implementation note found during code review

Team member: Gunnala Kavya — Extraction & Corpus (extract.py)

Observation for the team

In `extract.py` the import is written `from urlextract import Urlextract`, but the class in the installed `urlextract` 1.9.0 package is named `URLExtract` (capital U, L, E). The import therefore raises `ImportError`, which the code catches deliberately, so extraction silently runs on the built-in regular-expression fallback instead of the `urlextract` library. Results are still correct on the corpus, because the fallback matches `http`, `https` and `ftp` URLs. To use the library as documented, change the import and call to `URLExtract`. This document describes the behaviour as it actually runs.

Code Snippet 25 — *_build_extractor() — the import that falls back to the regex* [phishanalyzer/extract.py]

```
def _build_extractor():
# ...
    try:
        from urlextract import Urlextract

        return Urlextract()
    except Exception:  # noqa: BLE001 - any failure means "use the fallback"
        return None


_EXTRACTOR = _build_extractor()
_EXTRACTOR_BROKEN = False
```

11.3 Future enhancements

Team member: All members — joint section

- Independent DNS-based SPF evaluation, DKIM signature verification and DMARC policy retrieval.
- Domain-age / registration intelligence and typosquatting detection.
- Additional threat-intelligence providers and URL sandboxing.
- SIEM integration (for example Wazuh or Splunk) and a historical analyst dashboard.
- Evaluation on larger, properly licensed real-world corpora.
- Machine-learning-assisted classification as a research layer.

12. How to run the project

Team member: All members — joint section

Commands as documented in the project README (shown for Linux/macOS; on Windows activate the environment with `.venv\Scripts\activate`):

Code Snippet 26 — *Setup, analysis, evaluation, web UI and tests* [README.md]

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
pip install -e .

python tools/make_samples.py              # generate the synthetic corpus
python analyzer.py samples/synthetic      # analyse it -> output/
python tools/evaluate.py                  # metrics -> output/metrics.json
python analyzer.py mail.eml --virustotal  # opt in to live lookups (needs key)
```

```
python webapp.py                          # web UI at http://127.0.0.1:8000
pytest                                    # full test suite, network blocked
```

The VirusTotal key is optional. Copy `.env.example` to `.env` and set `VIRUSTOTAL_API_KEY`; without a key everything works except live reputation data.

Appendix A. Project structure

> **Team member: All members** — joint section

Code Snippet 27 — *Repository layout*

```
Phishing Email Analyzer/
  analyzer.py            command-line wrapper
  webapp.py              web launcher
  requirements.txt       pinned runtime dependencies
  pyproject.toml, README.md, INTERFACES.md, TEAM_ASSIGNMENTS.md
  phishanalyzer/
    models.py  parser.py  indicators.py  branding.py  refang.py
    extract.py  cli.py  webapp.py
    reputation/   base.py  offline.py  virustotal.py
    scoring/      weights.py  engine.py
    report/       html.py  text.py  csv_summary.py
    static/       bluecloud-logo.png
  tools/          make_samples.py  fetch_corpus.py  evaluate.py
  tests/          unit, contract, web and end-to-end tests
  samples/        synthetic corpus, ground_truth.csv, offline_blocklist.json
  output/         reports, summary.csv, results.json, metrics.json
```